Rapport de Projet

Sujet :

Conception et réalisation d'un moteur de recherche d'images par le contenu (CBIR) sur la base MIRFLICKR-25k

Descripteurs Couleur, LBP, HOG, MPEG-7 (EHD, HTD) et VGG16,
index VP-Tree, recherche textuelle et visuelle, application web FastAPI

Module : Indexation et recherche dans des bases massives MM

Réalisé par : Oussema Kard
Leith Mabrouk

Enseignant : M. Riadh Tébourbi

Année universitaire : 2026–2027

SUP'COM

Rapport de projet – 2026

Table des matières

Introduction générale **1**

1 Contexte et objectifs **2**
1.1 Principe d'un moteur CBIR . . . 2
1.2 Objectifs du projet . . . 2

2 Données utilisées **3**
2.1 La base MIRFLICKR-25k . . . 3
2.2 Annotations . . . 3
2.3 Descripteurs MPEG-7 fournis . . . 3

3 Architecture du système **5**
3.1 Vue d'ensemble . . . 5
3.2 Technologies utilisées . . . 6
3.3 Cohérence entre indexation et recherche . . . 6

4 Descripteurs visuels **7**
4.1 Histogramme de couleur HSV . . . 7
4.2 Motifs binaires locaux (LBP) . . . 7
4.3 Histogramme de gradients orientés (HOG) . . . 7
4.4 Descripteurs MPEG-7 : EHD et HTD . . . 8
4.5 Caractéristiques profondes VGG16 . . . 8
4.6 Images téléversées . . . 8

5 Méthodes de recherche **9**
5.1 Mesures de distance . . . 9
5.2 Combinaison de descripteurs . . . 9
5.3 Recherche exhaustive et VP-Tree . . . 9
5.4 Recherche par mots-clés . . . 9
5.5 Contrôle du flou . . . 10

6 Application web **11**
6.1 Le serveur . . . 11
6.2 L'interface . . . 11
6.3 Exemple : couleur contre VGG16 . . . 11

7 Évaluation **13**
7.1 Protocole . . . 13
7.2 Qualité de la recherche . . . 13
7.3 Rapidité : VP-Tree contre recherche exhaustive . . . 14

Conclusion **16**

Table des figures

3.1 Architecture générale du moteur de recherche 5

5.1 Recherche avec un flou de 0 % : seul le résultat le plus proche est conservé . . . 10
5.2 Même recherche avec un flou de 100 % : les K résultats demandés sont affichés . 10

6.1 Image requête #11120 (aigle) avec le descripteur Couleur seul 12
6.2 Même requête avec le descripteur VGG16 seul 12

7.1 Précision à 10 et à 20 pour chaque configuration, avec le classement aléatoire en pointillés 14
7.2 Part des distances calculées par le VP-Tree, en pourcentage de la recherche exhaustive 15
7.3 Temps de requête en fonction de la taille de la base, pour chaque descripteur . 15

Introduction générale

Les collections d'images numériques sont devenues si volumineuses qu'il est impossible de les parcourir à la main ou de les retrouver par leur seul nom de fichier. La recherche par mots-clés reste utile, mais les annotations sont coûteuses, incomplètes et subjectives : une même photographie peut être étiquetée « coucher de soleil », « sunset » ou ne pas être étiquetée du tout.

La recherche d'images par le contenu (*Content-Based Image Retrieval*, CBIR) décrit chaque image par des vecteurs numériques, appelés *descripteurs*, calculés directement à partir des pixels : couleurs, textures, contours ou caractéristiques apprises par un réseau de neurones. Chercher des images similaires revient alors à chercher les vecteurs les plus proches de celui de l'image requête. Deux questions se posent : celle de la **qualité** (le descripteur capture-t-il ce que l'utilisateur entend par « similaire » ?) et celle de la **rapidité** (comment éviter de comparer la requête à toutes les images de la base ?).

Dans le cadre du module *Indexation et recherche dans des bases massives MM*, nous avons réalisé un moteur de recherche textuelle et visuelle, avec une architecture client/serveur, sur la base MIRFLICKR-25k. Ce rapport présente les objectifs du projet, les données utilisées, l'architecture du système, les descripteurs, les méthodes de recherche, l'application web et l'évaluation des résultats.

Chapitre 1

Contexte et objectifs

1.1 Principe d'un moteur CBIR

Un moteur CBIR fonctionne en deux phases :

— une phase **hors ligne** (indexation), exécutée une seule fois : chaque image de la base est décrite par ses descripteurs, qui sont stockés avec une structure d'index accélérant les recherches ;

— une phase **en ligne** (recherche), exécutée à chaque requête : le système décrit l'image requête, la compare aux images de la base, puis renvoie les plus proches, classées par similarité décroissante.

1.2 Objectifs du projet

Le tableau 1.1 reprend les exigences de l'énoncé et la manière dont nous y avons répondu.

Tableau 1.1 : Exigences du projet et réponses apportées

Exigence	**Réponse**
Partie A : base MIRFLICKR avec les descripteurs MPEG-7 fournis	EHD et HTD chargés pour les 25 000 images
Partie B : descripteurs calculés	Couleur HSV, LBP, HOG, et en plus VGG16
Architecture client/serveur, échanges JSON	Serveur FastAPI, client HTML/CSS/JavaScript
Images dans des dossiers, URL en base de données	Images sur disque, chemins et tags dans SQLite
Requête par mots-clés ET/OU par image	Les deux, séparément ou combinées
Choix des descripteurs par l'utilisateur	Cases à cocher, choix de la métrique et du poids
Résultats classés par similarité décroissante	Image requête affichée, puis les résultats triés
Relancer une recherche depuis un résultat	Un clic sur un résultat relance la recherche
Temps de recherche optimal, structure d'index	Notre propre VP-Tree, comparé à la recherche exhaustive

Nous nous sommes également imposé deux contraintes : n'utiliser **aucun moteur de recherche externe** (Elasticsearch, Kibana, Logstash), afin d'écrire et d'étudier nous-mêmes la structure d'index, et faire fonctionner l'ensemble sur un ordinateur portable standard, **sans GPU**.

Chapitre 2

Données utilisées

2.1 La base MIRFLICKR-25k

La base MIRFLICKR, publiée par l'université de Leiden, rassemble des photographies issues de Flickr. Elle existe en deux versions : 25 000 images (environ 2,9 Go) et un million d'images (environ 120 Go). Les 25 000 premières images de la version 1M sont exactement celles de la version 25k.

Nous avons utilisé la version 25k : elle permet de calculer tous les descripteurs, y compris VGG16, sur un ordinateur sans GPU, et elle est fournie avec les annotations nécessaires à l'évaluation. L'archive contient les images `im1.jpg` à `im25000.jpg` ainsi que leurs tags Flickr, leurs données EXIF et leurs licences.

Tableau 2.1 : Chiffres clés de la base

Élément	**Valeur**
Images	25 000
Images ayant au moins un tag	22 872
Tags distincts	69 099
Concepts annotés (vérité terrain)	24
Images ayant au moins un concept	24 581

2.2 Annotations

Les annotations associent à chacune des 24 catégories (*people*, *sky*, *dog*, *car*, *sunset*…) la liste des images où elle apparaît. Elles servent uniquement de vérité terrain pour l'évaluation. Certaines catégories sont très larges : *people* couvre 10 373 images et *structures* 9 992, soit près de 40 % de la base chacune.

2.3 Descripteurs MPEG-7 fournis

MIRFLICKR fournit deux descripteurs MPEG-7 précalculés, sous forme d'un petit fichier texte par image. Chaque archive en contient un million ; nous avons écrit un court script qui n'extrait que les images 1 à 25 000. Faute de documentation, nous avons inspecté tous les fichiers avant de les utiliser :

— **EHD** (*Edge Histogram*) : 150 valeurs par image, ce qui correspond à la version étendue du descripteur ;

— **HTD** (*Homogeneous Texture*) : 43 valeurs par image, au lieu des 62 du format standard. Leur disposition n'étant pas documentée, nous traitons ce vecteur comme une signature de texture opaque.

Chapitre 3

Architecture du système

3.1 Vue d'ensemble

Le système suit les deux phases décrites au chapitre 1 (figure 3.1).

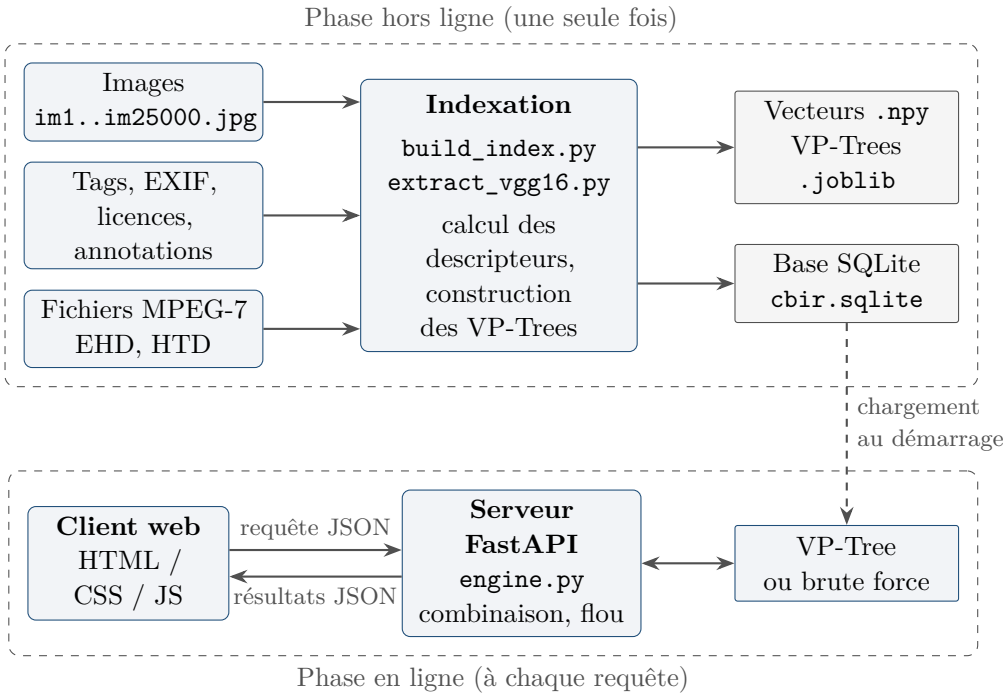


FIGURE 3.1 : Architecture générale du moteur de recherche

Phase hors ligne. Le script `build_index.py` calcule les descripteurs Couleur, LBP et HOG des 25 000 images en parallèle, lit les fichiers EHD et HTD, remplit la base SQLite (chemins, tags, EXIF, annotations) et construit les VP-Trees, en 7,8 minutes au total. Le script `extract_vgg16.py` calcule ensuite les caractéristiques VGG16 en 66 minutes sur CPU ; il peut être interrompu puis repris.

Phase en ligne. Au démarrage, le serveur charge en mémoire les vecteurs, les VP-Trees, la base SQLite et le réseau VGG16. Chaque requête est traitée en mémoire et le résultat est renvoyé au client en JSON. Les images restent dans leur dossier et sont servies à l'adresse `/images/im<id>.jpg`.

3.2 Technologies utilisées

Tableau 3.1 : Technologies retenues

Technologie	**Rôle**
Python 3.12, NumPy	Traitements, stockage des vecteurs, calcul des distances
OpenCV, scikit-image	Conversion HSV, calcul de LBP et HOG
PyTorch / torchvision	Réseau VGG16 pré-entraîné sur ImageNet
FastAPI	Serveur web et API JSON
SQLite	Chemins des images, tags, métadonnées et annotations
rapidfuzz	Tolérance aux fautes de frappe dans les mots-clés
HTML, CSS, JavaScript	Interface web, sans framework

Nous avons écarté Elasticsearch : il aurait pris en charge l'indexation à notre place, alors que la structure d'index est au cœur du module.

3.3 Cohérence entre indexation et recherche

Une image téléversée doit être décrite exactement comme les images de la base. Les paramètres de chaque descripteur sont enregistrés lors de l'indexation puis relus par le serveur, qui applique les mêmes fonctions. Nous l'avons vérifié en téléversant `im1.jpg` : l'image 1 est retrouvée en première position, à une distance nulle.

Chapitre 4

Descripteurs visuels

Tableau 4.1 : Récapitulatif des descripteurs

Descripteur	Origine	Capture	Taille	Métrique
Couleur (HSV)	calculé	distribution des couleurs	128	intersection
LBP	calculé	micro-texture	28	L1
HOG	calculé	forme, disposition des contours	3 780	L2
EHD (MPEG-7)	fourni	directions des contours	150	L1
HTD (MPEG-7)	fourni	texture	43	L1
VGG16	calculé (CNN)	contenu sémantique	512	L2

4.1 Histogramme de couleur HSV

L'image est convertie dans l'espace HSV (teinte, saturation, valeur), plus proche de la perception humaine que l'espace RGB. Nous calculons un histogramme de 128 cases, la teinte recevant le plus de cases car c'est elle qui porte l'identité de la couleur. L'histogramme est normalisé pour ne pas dépendre de la taille de l'image. Ce descripteur est simple et rapide, mais il ignore la disposition des couleurs et le contenu : une voiture rouge et une fleur rouge se ressemblent.

4.2 Motifs binaires locaux (LBP)

Chaque pixel est comparé à ses voisins répartis sur un cercle, ce qui donne un code binaire décrivant la texture locale. Nous utilisons la variante *uniforme invariante en rotation*, sur deux échelles (8 voisins à 1 pixel et 16 voisins à 2 pixels). L'histogramme des codes donne un vecteur compact de 28 valeurs.

4.3 Histogramme de gradients orientés (HOG)

L'image en niveaux de gris est redimensionnée à 64×128 pixels et découpée en cellules de 8×8 pixels. Chaque cellule accumule un histogramme de 9 orientations de gradient ; les cellules sont regroupées en blocs normalisés, ce qui rend le descripteur robuste aux variations d'éclairage. HOG décrit la forme globale de l'image en 3 780 valeurs.

4.4 Descripteurs MPEG-7 : EHD et HTD

L'**EHD** découpe l'image en 4×4 régions et compte, dans chacune, cinq types de contours : vertical, horizontal, deux diagonales et non directionnel. Le **HTD** décrit la texture à partir de filtres de Gabor à plusieurs échelles et orientations. Les deux sont utilisés tels que fournis ; les valeurs du HTD, d'ordres de grandeur très différents, sont simplement remises à la même échelle.

4.5 Caractéristiques profondes VGG16

VGG16 est un réseau de neurones convolutif entraîné sur ImageNet (1,2 million d'images, 1 000 classes). Nous utilisons sa partie convolutive pré-entraînée : l'image, redimensionnée à 224×224, traverse le réseau, et la moyenne des dernières cartes de caractéristiques donne un vecteur de 512 valeurs. Contrairement aux descripteurs précédents, conçus à la main, ces caractéristiques encodent un **contenu sémantique** (objets, animaux, types de scènes).

4.6 Images téléversées

Couleur, LBP, HOG et VGG16 sont calculés à la volée pour une image téléversée. EHD et HTD, en revanche, n'existent que pour les images de la base : une version calculée par nos soins ne serait pas compatible avec les fichiers fournis. Ces deux descripteurs sont donc désactivés dans l'interface pour les images téléversées.

Chapitre 5

Méthodes de recherche

5.1 Mesures de distance

Deux descripteurs sont comparés par l'une de trois distances : L1 (somme des écarts), L2 (distance euclidienne) ou intersection d'histogrammes. Pour des histogrammes normalisés, l'intersection donne exactement le même classement que L1. Les trois respectent l'inégalité triangulaire, condition nécessaire au fonctionnement du VP-Tree.

5.2 Combinaison de descripteurs

L'utilisateur peut cocher plusieurs descripteurs et donner un poids à chacun. Comme leurs distances n'ont pas la même échelle (environ 0,8 pour la couleur, environ 42 pour le HTD), chaque distance est d'abord ramenée à une échelle commune, puis les distances sont moyennées selon les poids choisis. Ainsi, aucun descripteur ne domine les autres simplement à cause de ses unités.

5.3 Recherche exhaustive et VP-Tree

La **recherche exhaustive** (*brute force*) compare la requête aux 25 000 images, puis garde les plus proches. Elle est vectorisée avec NumPy et sert de référence.

Le **VP-Tree** (*Vantage-Point Tree*) est notre propre structure d'index. À chaque nœud, il choisit une image de référence et répartit les autres en deux groupes : celles proches de la référence et celles éloignées. Lors d'une recherche, l'inégalité triangulaire permet de savoir qu'un groupe entier est trop loin de la requête pour contenir un bon résultat ; ce groupe est alors ignoré sans calculer ses distances. La recherche reste **exacte** : elle renvoie toujours les mêmes résultats que la recherche exhaustive, ce que nous avons vérifié par un test automatique et sur toutes les requêtes de l'évaluation.

Nos premiers arbres, avec des feuilles de 32 images, étaient plus lents que la recherche exhaustive : chaque nœud visité exécute du code Python, alors que la recherche exhaustive est une seule opération NumPy. Avec des feuilles de 512 images, le VP-Tree devient plus rapide pour les descripteurs de petite taille.

5.4 Recherche par mots-clés

Les mots-clés sont comparés aux tags Flickr stockés dans SQLite, en mode ET (toutes les images ayant tous les mots-clés) ou OU (au moins un). Pour tolérer les fautes de frappe, chaque

mot-clé est rapproché des tags les plus proches : par exemple, `sunst` est reconnu comme `sunset`. Lorsqu'un mot-clé et une image sont fournis ensemble, les mots-clés filtrent d'abord la base, puis les images retenues sont classées par similarité visuelle.

5.5 Contrôle du flou

Un curseur de « flou » (*fuzziness*), de 0 à 100 %, règle la sévérité de la recherche :

— en recherche visuelle, à 100 % on obtient les K résultats habituels ; plus la valeur baisse, plus seuls les résultats très proches du meilleur sont conservés. Par exemple, avec VGG16 et 20 résultats demandés, on obtient 20 résultats à 100 %, 8 à 50 % et 2 à 20 % ;

— en recherche par mots-clés, à 0 % seuls les tags identiques sont acceptés ; plus la valeur augmente, plus les fautes de frappe sont tolérées.

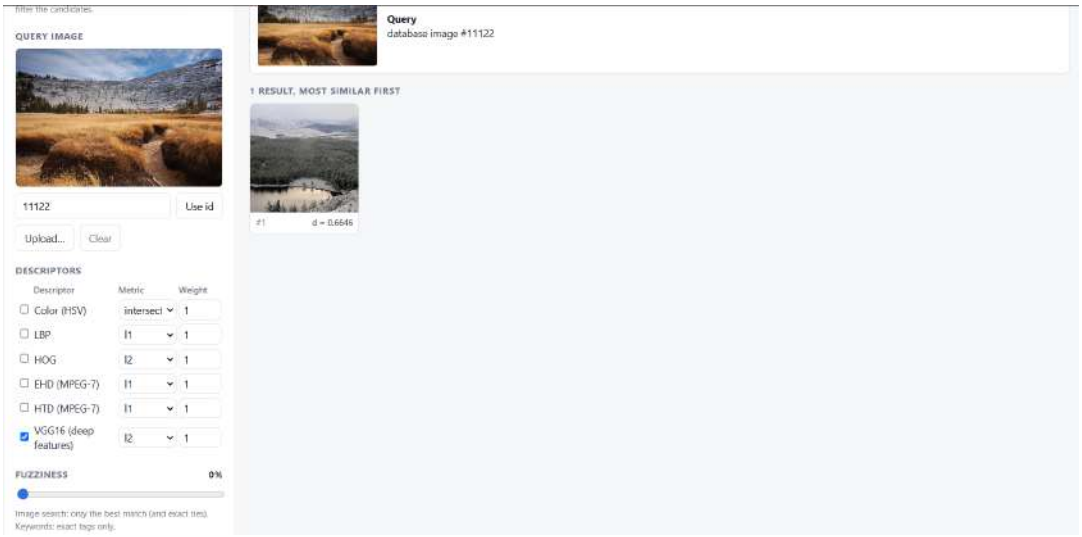


FIGURE 5.1 : Recherche avec un flou de 0 % : seul le résultat le plus proche est conservé

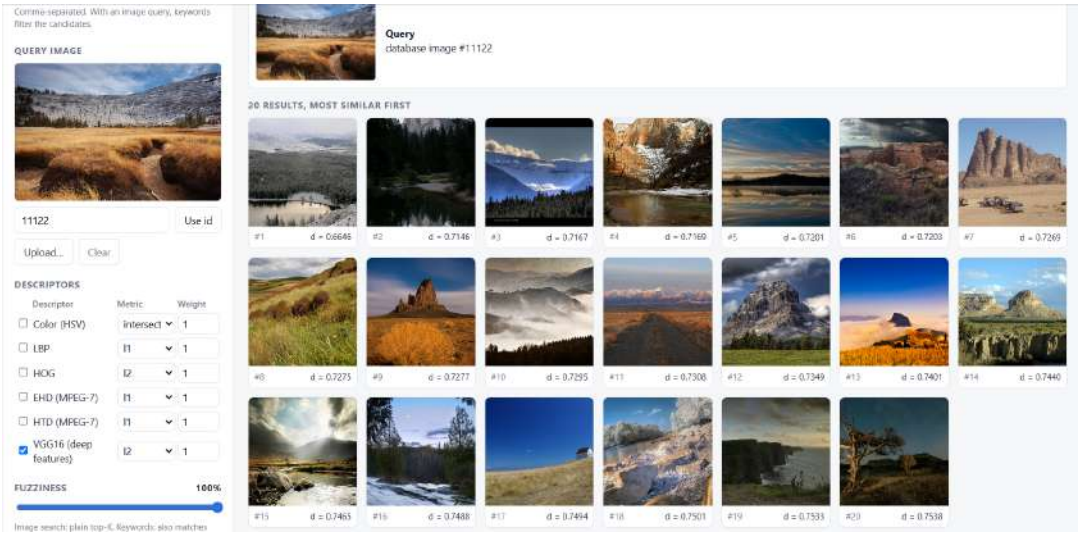


FIGURE 5.2 : Même recherche avec un flou de 100 % : les K résultats demandés sont affichés

Chapitre 6

Application web

6.1 Le serveur

Le serveur FastAPI expose une API JSON. Les routes principales sont `POST /api/search` (recherche par image de la base et/ou par mots-clés) et `POST /api/search/upload` (recherche avec une image téléversée). D'autres routes fournissent la configuration, des images aléatoires pour la galerie, les métadonnées d'une image et l'autocomplétion des tags.

6.2 L'interface

L'interface est une page web unique. Le panneau latéral contient les mots-clés, le choix de l'image requête (identifiant, galerie ou téléversement), le tableau des descripteurs (case à cocher, métrique, poids), le curseur de flou, le choix de l'index (VP-Tree ou brute force) et le nombre de résultats. La zone principale affiche l'image requête, puis les résultats du plus similaire au moins similaire avec leur distance. Une barre d'état indique le temps de recherche, l'index utilisé et le nombre de distances calculées. Un clic sur un résultat relance la recherche avec cette image comme requête.

6.3 Exemple : couleur contre VGG16

Pour illustrer l'influence du descripteur, nous interrogeons la même image, un aigle (image #11120), avec deux descripteurs différents. Avec l'histogramme de couleur seul (figure 6.1), les résultats sont des images à dominante orangée ou brune, la couleur du fond, sans lien avec l'aigle. Avec VGG16 seul (figure 6.2), les résultats sont des oiseaux : le réseau reconnaît le contenu de l'image, pas seulement ses couleurs.

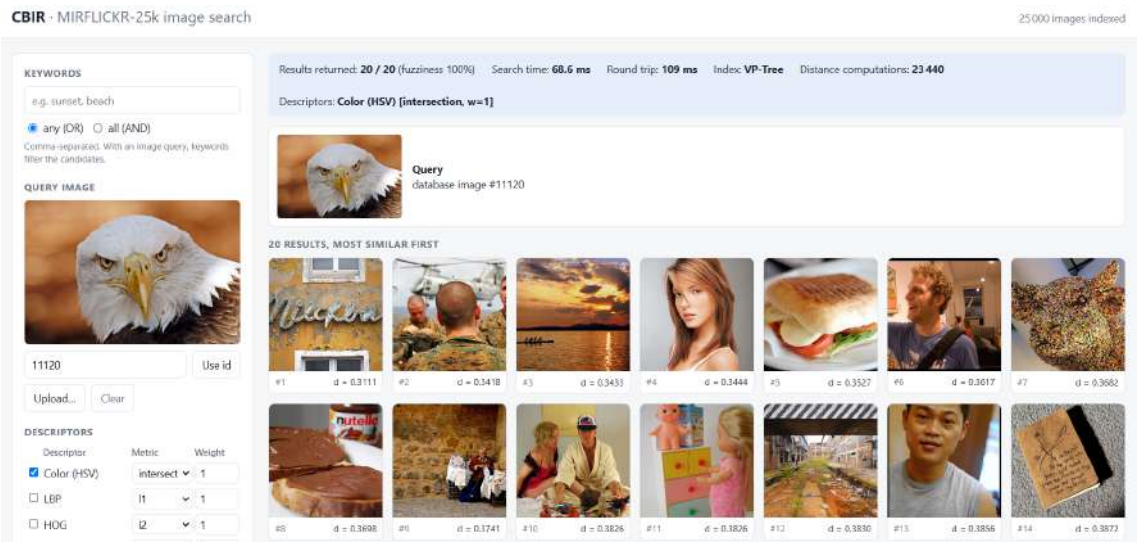


FIGURE 6.1 : Image requête #11120 (aigle) avec le descripteur Couleur seul

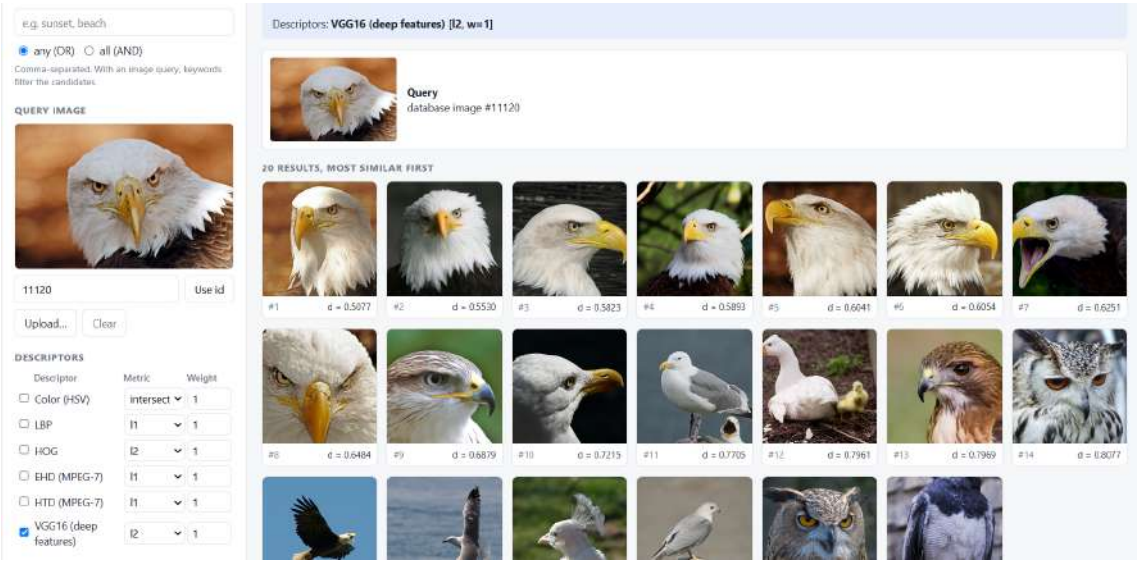


FIGURE 6.2 : Même requête avec le descripteur VGG16 seul

Chapitre 7

Évaluation

7.1 Protocole

Nous avons évalué le moteur sur 100 images requêtes tirées au hasard. Un résultat est considéré comme **pertinent** s'il partage au moins une catégorie annotée avec la requête. Nous mesurons la précision à 10 et à 20 (proportion de résultats pertinents parmi les 10 ou 20 premiers) et la précision moyenne (mAP), qui récompense les bons résultats placés en tête.

Les catégories étant très larges, un classement **aléatoire** obtient déjà une précision de **0,564** : les résultats doivent être lus par rapport à cette valeur.

7.2 Qualité de la recherche

Tableau 7.1 : Qualité de la recherche (100 requêtes, 25 000 images)

Configuration	Précision à 10	Précision à 20	mAP à 20
VGG16	**0,892**	**0,888**	**0,841**
Couleur + LBP + VGG16	0,831	0,816	0,728
Couleur + LBP + HOG	0,727	0,712	0,591
LBP	0,697	0,671	0,535
Couleur	0,650	0,646	0,499
EHD	0,578	0,577	0,399
HTD	0,544	0,552	0,372
HOG	0,508	0,508	0,345
Classement aléatoire	*0,564*	*0,564*	—

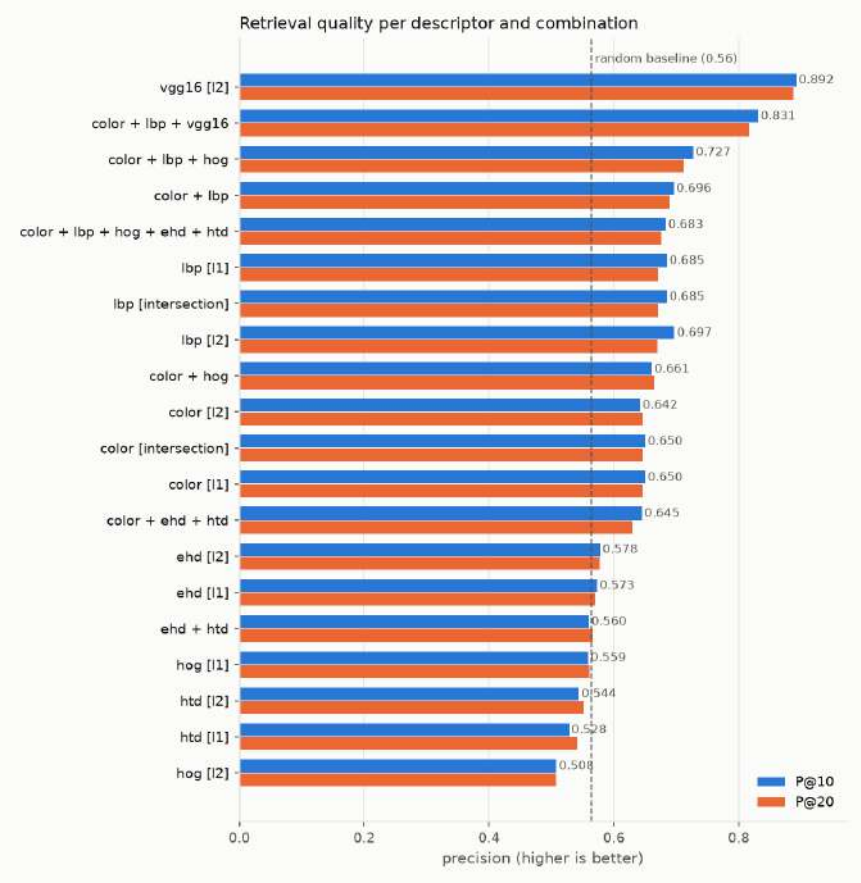


FIGURE 7.1 : Précision à 10 et à 20 pour chaque configuration, avec le classement aléatoire en pointillés

— **VGG16 domine nettement** (0,892) : les catégories annotées sont sémantiques (chien, voiture, personnes) et VGG16 a été entraîné à reconnaître ce type de contenu.

— **LBP et la couleur** sont les meilleurs descripteurs classiques (0,65 à 0,70). EHD dépasse à peine le hasard, et HOG comme HTD restent à son niveau.

— **Combiner aide les descripteurs classiques** (Couleur + LBP + HOG fait mieux que chacun seul), **mais nuit à VGG16** : lui ajouter des descripteurs plus faibles fait baisser sa précision de 0,892 à 0,831.

7.3 Rapidité : VP-Tree contre recherche exhaustive

TABLEAU 7.2 : Temps moyen d'une requête et part des distances calculées par le VP-Tree

Descripteur (taille)	VP-Tree	Brute force	Distances calculées	Exact
LBP (28)	0,6 ms	0,9 ms	21 %	oui
HTD (43)	4,3 ms	2,9 ms	38 %	oui
Couleur (128)	4,2 ms	5,4 ms	82 %	oui
EHD (150)	13,1 ms	19,4 ms	61 %	oui
VGG16 (512)	34,7 ms	33,4 ms	100 %	oui
HOG (3 780)	393 ms	214 ms	100 %	oui

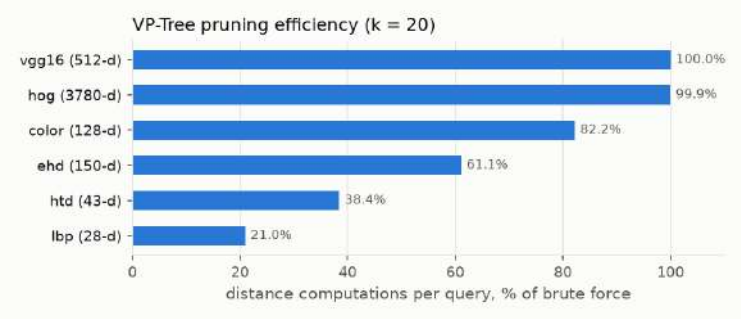


FIGURE 7.2 : Part des distances calculées par le VP-Tree, en pourcentage de la recherche exhaustive

— Le VP-Tree est **toujours exact** : il renvoie les mêmes résultats que la recherche exhaustive.

— Pour les petits descripteurs, il évite une grande partie des calculs : LBP n'en effectue que 21 %. Pour VGG16 et HOG, il les effectue tous. C'est la **malédiction de la dimension** : en grande dimension, toutes les distances se ressemblent et l'arbre ne peut plus écarter de groupes d'images.

— Moins de calculs ne signifie pas toujours moins de temps : notre arbre est écrit en Python, alors que la recherche exhaustive est une opération NumPy très optimisée. Le VP-Tree gagne pour EHD, LBP et la couleur, et perd lorsqu'il ne peut rien écarter.

— Son avantage augmente avec la taille de la base : pour LBP, la part des distances calculées passe de 73 % avec 1 000 images à 18 % avec 25 000.

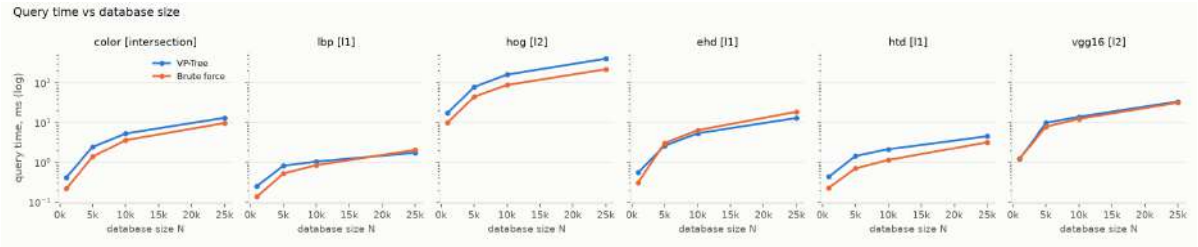


FIGURE 7.3 : Temps de requête en fonction de la taille de la base, pour chaque descripteur

Conclusion

Nous avons réalisé un moteur complet de recherche d'images par le contenu sur les 25 000 images de MIRFLICKR-25k : six descripteurs visuels, dont deux descripteurs MPEG-7 fournis et quatre calculés par nos soins, notre propre index VP-Tree, un serveur FastAPI, une interface web, une recherche par mots-clés tolérante aux fautes de frappe, la combinaison pondérée des descripteurs et un contrôle du flou.

L'évaluation montre que les caractéristiques apprises (VGG16, précision à 10 de 0,892) surpassent nettement les descripteurs conçus à la main (au mieux 0,70) pour retrouver des images de même contenu. Le VP-Tree est exact et efficace pour les descripteurs de petite taille, mais il ne peut rien écarter pour les descripteurs de grande taille comme HOG et VGG16.

Plusieurs améliorations sont possibles : une version compilée du VP-Tree pour transformer la réduction des calculs en gain de temps réel, une réduction de la taille de VGG16 et HOG avant indexation, le passage à la base complète MIRFLICKR-1M, et l'indexation de la base « Objects », que le code permet déjà grâce à des chemins configurables.

Le code source est disponible à l'adresse `https://github.com/kard4318/cbir-mirflickr`, et une vidéo de démonstration accompagne ce rapport.